

Design Demands For Android application

While developing an android application for mobile devices we need to consider following design consideration and design demands in mind.

1. **It should be fast and efficient** : You should optimize the code of your android application so that it runs fast and efficient even if the hardware associated with it changed.
2. **It should use less storage as possible** : An android application should use optimal amount of storage space and resources as much as possible. It should incorporate some cleaning mechanism by closing and freeing resources that are no longer needed.
3. **It should present consistent UI for all devices** : The UI of the application should be designed such that it provides consistent look for both large and small screen.
4. **It should be able to handle low speed and high latency** : While developing an internet based application you have to assume that the speed of the internet is not equal all the time. So you have to develop an application by considering slow internet data connection in your mind.
5. **It should be secure**: Hence android application can have direct access to hardware and it is distributed independently and developed on open source platform it should provide security restrictions.
6. **It should ensure look and feel environment to its user** : The layout of an android application should be designed such that it provides look and feel environment to its users.
7. **It should be responsive** : An android application should use worker threads and background services to keep your activities responsive.

Design considerations for mobile computing

The **mobile computing** environment is constrained in many ways. Mobile elements themselves are resource-poor and unreliable. Their network connectivity is often achieved through low-bandwidth wireless links.

These severe restrictions have a great impact on the design and structure of mobile computing applications and motivate the development of new computing models.

These **mobile computing models must provide** efficient access to both existing and new applications which is a key requirement for the wide acceptance of mobile computing.

An **important design consideration is the type of functionality assigned to mobile hosts**. Mobile units are still characterized as unreliable and prone to hard failures, i.e., theft, loss or accidental damage.

Thus following requirements must be considered for mobile computing environment:

1. Type of Application (Native or Mobile Web) :

- Before designing mobile application you need to determine type of application.
- If your application requires local processing. access to local resources than consider designing a native application.
- A native application is hard to maintain. requires separate distribution and upgrade infrastructure, are compatible only with target device/platform, requires more effort (sometimes huge) to port on different devices.
- A mobile web application is compatible with all devices with internet connection and a browser

2. Target device :

- Target device and platform (OS) plays a key role throughout design decisions making process.
- Design decisions are influenced by target device's screen size, resolution, orientations, memory, CPU performance characteristics. Operating systems capabilities, device hardware, user input mechanism (touch/non-touch), sensors (such as GPS or accelerometer etc).

3. User experience:

- User interface should be rich, intuitive and responsive. While using mobile application user is often distracted by external or internal (e.g. incoming call when user is in middle of a wizard) events. These events have to be factored in design.
- Screens should be identified with consideration that user is often focused on discrete individual task. Long data entry forms may irritate user and should ask for absolutely required data/information.

4. Resource Constraint:

- Every design decision should take into account the limited CPU, memory and battery life. Reading and writing to memory, wireless connections, specialized hardware, and processor speed all have an impact on the overall power usage.
- One cannot afford a long running process in application.
- In some cases, application processing should be offloaded from the device to avoid excessive resource consumption e.g. using notification or app directed SMS instead of polling to monitor a value/flag on server.

5. Multiple Platforms:

- There is high technology fragmentation in Mobile world. It is very unlikely that an application will target only one platform or only one device. In near future, requirement like same code base should support iPhone and iPad or Android Phone and Android tablet will arise. Architect should consider portability as important architectural goal.
- If you are developing for more than one device, design first for the subset of functionality that exists on all of the devices, and then customize for device-specific features when they are detected.

6. Security:

- Devices are more vulnerable than desktop, primarily due to lack of awareness. Moreover device can be lost easily. Ensure that the device – server communication is secured and server accepts request only from authentic source (device).
- If you are storing any confidential application or configuration data locally. ensure that the data is encrypted.

7. Network Communication:

- Network communication on device is slow and costly. Reduce network traffic by combining several commands in one request e.g. committing added, updated and deleted record in one request instead of firing separate request on each add/update/delete.
- Compress large text / XML data to lesser network traffic. Design for asynchronous communication.

Introduction to Activities in Android

1. Activity class is one of the very important parts of the [Android Component](#). Any app, don't matter how small it is (in terms of code and scalability), has at least one Activity class.
2. Unlike most programming languages, in which the **main()** method is the entry point for that program or application to start its execution, the android operating system initiates the code in an Activity instance by invoking specific callback methods that correspond to specific stages of its [Lifecycle](#).
3. As we know that every activity contains a layout associated with it, so it can be said that the activity class is the gateway, through which a user can interact programmatically with the UI. Layout for a particular activity is set with the help of **setContentView()**. **setContentView()** is a function that takes **View** as a parameter. The view parameter basically contains the layout file for that activity. The code has been given in both **Java and Kotlin Programming Language for Android**.

activity_main is the Layout File of MainActivity

```
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {

    @Override
```

```

        protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);
        }
    }

```

While activities are often presented to the user as the full-screen window, Multiwindow mode, or Picture in Picture mode, here are two methods almost all subclasses of Activity will implement:

1. **onCreate()**
2. **onPause()**

The syntax for Java:

```
protected void onCreate(Bundle savedInstanceState)
```

- **onCreate(Bundle) method** is the place, where the user initializes the activity. This method is called when the activity is starting. This is the method that is used to initialize most of the things in the android app. **onCreate()** method takes **savedInstanceState** as the parameter, which is the object of type Bundle, i.e Bundle Object which contains the previously saved data of the activity. If the activity is newly created then the bundle won't be saving any data of the activity and would contain the null value.
- **onCreate() method** calls the **setContentView()** method to set the view corresponding to the activity. By default in any android application, setContentView point to activity_main.xml file, which is the layout file corresponding to MainActivity. The onCreate method uses the **findViewById()** method so that the user can interact programmatically with the widgets in android and then customize them according to need.

Bundle: If the activity is being re-initialized or restarted after previously being closed, then this Bundle contains the data it most recently supplied in **onSaveInstanceState(Bundle)**. **onSaveInstanceState()** method is the method that is called to save the data just before our activity is killed.

2. onPause() Method

The syntax for Java:

```
protected void onPause() {
    super.onPause();
}

```

This method is called part of the [Activity Lifecycle](#) when the user no longer actively interacts with the activity, but it is still visible on the screen. Let's suppose the user is running two apps simultaneously on a mobile phone, i.e when activity B is launched in front of activity A, activity A will go in the **onPause()** state, and activity B would go in the **onStart()** state of the activity lifecycle.

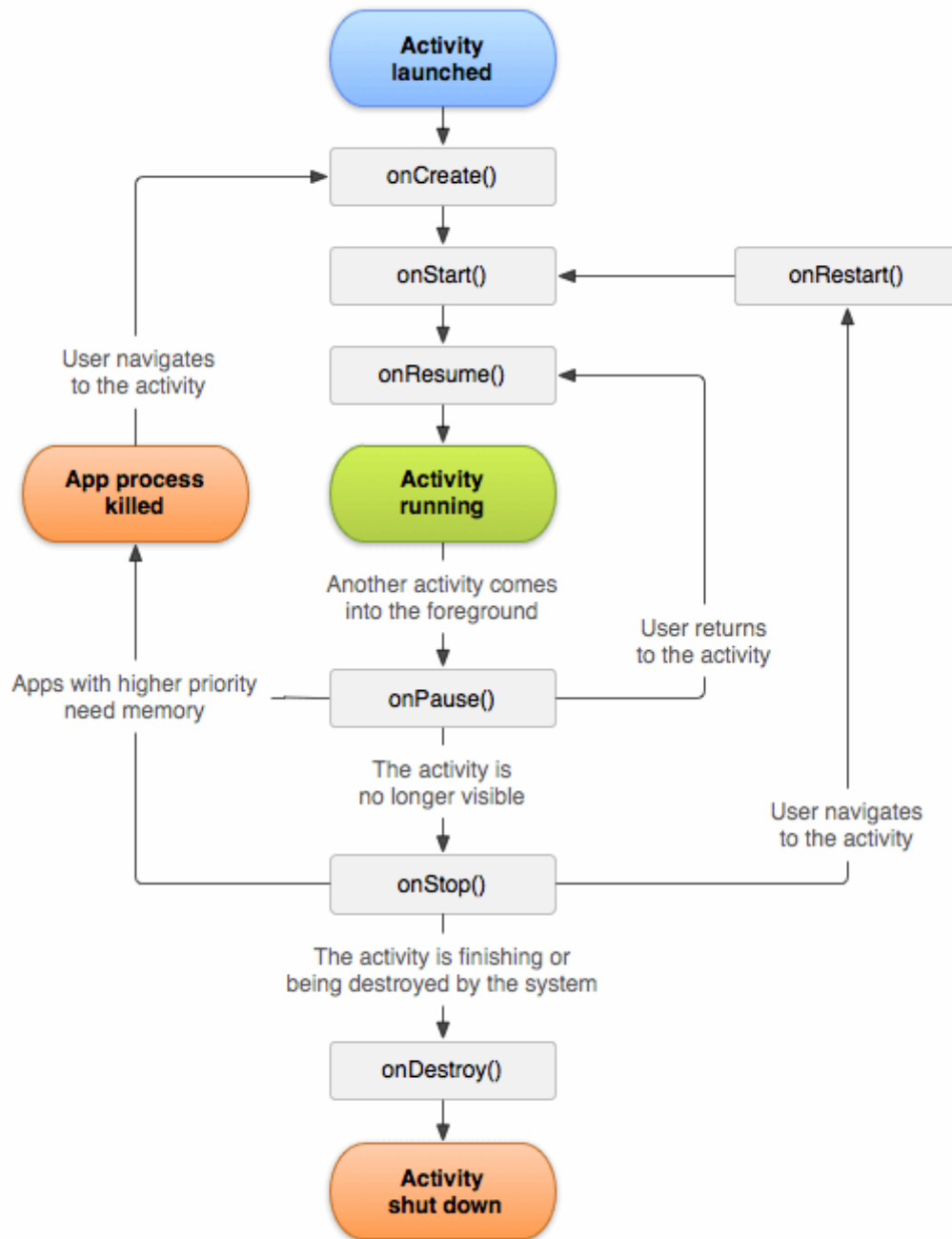
Android Activity Lifecycle

An activity is the single screen in android. It is like window or frame of Java.

By the help of activity, you can place all your UI components or widgets in a single screen.

The 7 lifecycle method of Activity describes how activity will behave at different states.

Method	Description
onCreate	called when activity is first created.
onStart	called when activity is becoming visible to the user.
onResume	called when activity will start interacting with the user.
onPause	called when activity is not visible to the user.
onStop	called when activity is no longer visible to the user.
onRestart	called after your activity is stopped, prior to start.
onDestroy	called before the activity is destroyed.



Layout in Android:

Layout – Layout are used to define the actual User interface of application. It holds all the elements (i.e. views) or the tools that we want to use in application. For example, `ImageView`, `Button`, `TextView` and other UI elements.

Types Of Layout And Few Important ViewGroup In Android

1. Linear Layout
2. Relative Layout

3. Table Layout
4. FrameLayout
5. Absolute Layout
6. ConstraintLayout

7. WebView

8. ListView
9. GridView

Recycler View

Different Attribute of the Layouts

XML attributes	Description
android:id	Used to specify the id of the view.
android:layout_width	Used to declare the width of View and ViewGroup elements in the layout.
android:layout_height	Used to declare the height of View and ViewGroup elements in the layout.
android:layout_marginLeft	Used to declare the extra space used on the left side of View and ViewGroup elements.
android:layout_marginRight	Used to declare the extra space used on the right side of View and ViewGroup elements.
android:layout_marginTop	Used to declare the extra space used in the top side of View and ViewGroup elements.
android:layout_marginBottom	Used to declare the extra space used in the bottom

	side of View and ViewGroup elements.
android:layout_gravity	Used to define how child Views are positioned in the layout.

Introduction to Android UI Design:

The Android operating system (OS) allows users to interact with it through different devices and a medium called the User Interface (UI).

The **UI** is what the end user is able to see and interact with to navigate and use different components of the Android OS on an Android application. They are presented to the user in different forms, such as the following:

- **Graphical User Interface (GUI):** This helps users interact with visual representations on digital devices or smartphones such as a Samsung phone.
- **Voice-controlled interfaces:** These allow users to interact through voice commands, such as Siri and Alexa.
- **Gesture-based interface:** Users are able to interact with the interface through body motions.

What is Android UI design?

Android UI design involves the use of prebuilt Android UI components, such as structured layout objects and UI controls, to build the graphical user interface for our applications.

An Android App UI screen usually consists of four parts, which are as follows:

- Status bar
- App bar
- Content area
- Bottom navigation bar

The Android OS UI components consists of different types of layouts, and special interfaces such as menus, notifications, and dialogs.

External resources in Android

Android UI Design allows external files and elements such as images, vector graphics, SVG to be used in the UI design to beautify it more for great user experience. Such files are kept in a folder called `drawable` in the Android studio.

Measurements in Android UI Design

We use the **DP** and **SP** to define measurement in Android UI. The DP is used to define UI events such as height and weight. The SP is used to define text size so that it is consistent across different devices. It also allows the user to choose their preferred size on their devices.

Push Button

In Android, **Button** represents a push [button](#). A Push buttons can be clicked, or pressed by the user to perform an action. There are different types of buttons used in android such as `CompoundButton`, [ToggleButton](#), [RadioButton](#).

Attributes of Button in Android:

1. **id:** id is an attribute used to uniquely identify a text Button.
2. **gravity:** The gravity attribute is an optional attribute which is used to control the alignment of the text like left, right, center, top, bottom, center_vertical, center_horizontal etc.
3. **text:** text attribute is used to set the text in a Button.
4. **textColor:** textColor attribute is used to set the text color of a Button.
5. **textSize:** textSize attribute is used to set the size of the text on Button.
6. **textStyle:** textStyle attribute is used to set the text style of a Button.
7. **background:** background attribute is used to set the background of a Button.
8. **padding:** padding attribute is used to set the padding from left, right, top or bottom.
9. **drawableBottom:** drawableBottom is the drawable to be drawn to the below of the text.

10. drawableTop, drawableRight And drawableLeft: Just like the above attribute we can draw drawable to the left, right or top of text.

Textview:

In Android, [TextView](#) displays text to the user and optionally allows them to edit it programmatically.

Attributes of TextView:

1. **id:** id is an attribute used to uniquely identify a [text view](#).
2. **gravity:** The gravity attribute is an optional attribute which is used to control the alignment of the text like left, right, center, top, bottom, center_vertical, center_horizontal etc.
3. **text:** text attribute is used to set the text in a text view.
4. **textColor:** textColor attribute is used to set the text color of a text view.
5. **textSize:** textSize attribute is used to set the size of text of a text view.
6. **textStyle:** textStyle attribute is used to set the text style of a text view.
7. **background:** background attribute is used to set the background of a text view.
8. **padding:** padding attribute is used to set the padding from left, right, top or bottom.

Android Edit Text

EditText is a *Widget of user interface* (UI) used to retrieve and modify text data from a user in an Android app.

XML Attributes of EditText

Attributes	Description
android:id	It is used to uniquely identify the element
android:width	It is used to set the width of EditText in pixels
android:height	It is used to set the height of EditText in pixels

android:text	It is used to set the text value inside the EditText
android:inputType	It is used to mention what type of value should pass in EditText, like plain text, email, password, etc
android:hint	It is used to highlight what should be input when text is empty
android:gravity	gravity is used to align the text like left, right, center, top
android:textSize	It is used to specify the size of the text
android:textStyle	It is used to set the text style like bold, italic, bold italic, etc.
android:textColor	It is used to set the color of the text
android:background	It is used to set the background of your EditText
android:backgroundTint	It is used to set tint to the background of your element
android:maxLength	It is used to set the maximum width of View in pixel
android:minWidth	It is used to set the minimum width of View in pixel
android:drawableStart	It is used to set the drawable to be drawn to the start of View
android:drawableEnd	It is used to set the drawable to be drawn to the end of View
android:drawableBottom	It is used to set the drawable to be drawn to the bottom of View
android:drawableLeft	It is used to set the drawable to be drawn to the left of View
android:drawableRight	It is used to set the drawable to be drawn to the right of View
android:drawablePadding	It is used to set the drawable to be drawn from the set padding of View

Android ToggleButton

Android Toggle Button can be used to display checked/unchecked (On/Off) state on the button.

android:textOff	The text for the button when it is not checked.
android:textOn	The text for the button when it is checked.